Software Engineering Educational Experience in Building an Intelligent Tutoring System

Zhiyu Fan
zhiyufan@comp.nus.edu.sg
National University of Singapore
Singapore

Yannic Noller*
yannic.noller@acm.org
Singapore University of Technology and Design
Singapore

Ashish Dandekar
ashishd@comp.nus.edu.sg
National University of Singapore
Singapore

Abhik Roychoudhury
abhik@comp.nus.edu.sg
National University of Singapore
Singapore

ABSTRACT

The increasing number of computer science students pushes lecturers and tutors of first-year programming courses to their limits. Existing systems that handle automated grading primarily focus on the automation of test case executions in the context of programming assignments. However, they cannot provide customized feedback about the students' errors, and hence, cannot replace the help of tutors. Based on the research advances in recent years, specifically in automated program repair and synthesis, we have built an intelligent tutoring system that has the capability to provide automated feedback. Furthermore, we designed a software engineering course that guides third-year undergraduate students in incrementally developing such a system over several years. Each year, students will make contributions that improve the current implementation, while at the same time, we can deploy the current system for first-year students to use for learning programming. This paper describes our teaching concept, the intelligent tutoring system architecture, and our experience with the stakeholders. This software engineering project for the students has the key advantage that the users of the system are available in-house (i.e., students, tutors, and lecturers from the first-year programming courses). This helps to organize requirements engineering sessions and builds awareness about their contribution to a "to–be–deployed" software project. In this multi-year teaching effort, we have incrementally built a tutoring system for first-year programming courses.

CCS CONCEPTS

• **Software and its engineering**; • **Applied computing** → **Education**;

KEYWORDS

computer science education, automated program repair, intelligent tutor

*This work was done when all four authors were at National University of Singapore.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ICSE-SEET '24, April 14–20, 2024, Lisbon, Portugal
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0498-7/24/04.
https://doi.org/10.1145/3639474.3640049

ACM Reference Format:
Zhiyu Fan, Yannic Noller, Ashish Dandekar, and Abhik Roychoudhury. 2024. Software Engineering Educational Experience in Building an Intelligent Tutoring System. In *46th International Conference on Software Engineering: : Software Engineering Education and Training (ICSE-SEET '24), April 14–20, 2024, Lisbon, Portugal.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3639474.3640049

1 INTRODUCTION

In Computer Science (CS) education, we face the issue of the increasing number of enrolments in the past years [20]. Therefore, it becomes increasingly difficult to keep up with high-quality and individual learning support, particularly for novice students [17, 27]. Mirhosseini et al. [17] recently conducted an interview study with CS instructors to identify their biggest *pain points*. Amongst others, they identified that CS instructors struggle with no or limited Teaching Assistant (TA) support and the generally time-consuming task of providing student feedback and assignment grading. Therefore, CS instructors would greatly benefit from automating tutoring activities to support TAs in their responsibilities. Another typical problem in CS education is the provision of Software Engineering (SE) projects. Software engineering is typically a compulsory course in the university's curriculum for computer science students, and it is often followed or accompanied by development projects, in which students can collect hands-on experience in software development in a team going beyond a programming exercise. Such projects come with inherent difficulties like acquiring industry partners and the dilemma that such software projects are often under- or over-specified. Additionally, such projects are often one-time efforts within one team or one course, and students cannot experience the evolution of a software system.

In this work, we report our experience in tackling these two problems in CS education by building an *Intelligent Tutoring System* (ITS) *with* and *for* students. As a multi-year research and teaching effort, we combine third-year SE teaching and first-year programming teaching via a long-term, practical, self-sustained software system. We use the latest research results in Automated Program Repair (APR), e.g., techniques like Clara [10], SarfGen [24], and Refactory [11], to build such intelligent tutoring system that can be deployed in first-year programming courses.

Figure 1 shows the general idea of such a system. It can provide automated and individual feedback for student code submissions

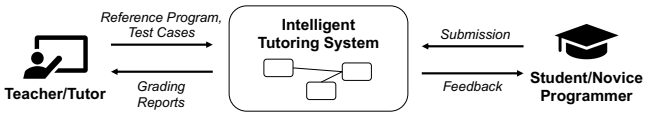


Figure 1: General idea of an intelligent tutoring system that supports students and tutors in CS-1 programming courses.

and grading support for tutors and lecturers. Further, we involve third-year undergraduate students in the incremental development of such a system. We offer various SE projects for the students in our advanced SE course. In this course, the students can choose from a wide range of projects, which essentially represent the development or extension of ITS components.

Based on the nature of the overall project, we can conduct requirements engineering activities (e.g., surveys, interviews, and user studies) in-house because the various stakeholders are available in the university context. Each student project has the chance to contribute to the overall long-running SE project and eventually impact the learning experience of hundreds of other CS students. In our experience, this creates additional motivation because the effort is not lost, and they can relate to the users because they (at some point in their studies) also faced the challenges of learning programming. Based on our experience with around 125 undergraduate students who helped develop the system throughout two years of teaching, the students enjoyed the course project. In particular, they liked the potential reuse of their implementation in the real deployment of the ITS. Additionally, they enjoyed the fact that there is already a system, which they have to extend including the added complexity in understanding the already existing architecture, design, and codebase.

Our user studies revealed that in the current version of the ITS, the students and tutors benefit the most from the error localization capabilities, which pinpoint the student's error to the specific lines in their submission. It helps the students find their errors and allows the tutors quickly understand the problems in the student submissions. In particular, for the students, the auto-generated feedback can help them to understand their problems and correct their mistakes. The students reported that they see the potential of ITS to provide automated, and hence, always available feedback. We also received new requirements that can further help to improve the current implementation, e.g., more error explanations and integration into other learning platforms, as well as feedback about non-functional properties and advanced visualization and interaction features. Overall, more than 78% of our participants would like the ITS to be deployed in their next programming course.

Throughout the two years we have taught this SE course, we learned that students prefer certain types of projects (e.g., in our case, they preferred front-end projects and avoided the projects about core APR capabilities), which influenced how we organized the project selection. Further, we noticed that students benefit from the additional help from graduate-level mentors. Our course not only impacts the first-year programming courses in our university but also has the potential to impact other universities which adopt a similar teaching concept linking the teaching of software engineering with the teaching of programming. In the future, we plan to conduct more user studies to explore learning success across university boundaries.

In summary, we make the following core contributions:

- We present our teaching concept that involves the incremental development of an Intelligent Tutoring System (ITS) spanning multiple semesters.
- We share our experience with the user (i.e., student) engagements, where more than 78% of them want to see the ITS deployed in their next programming course.
- We suggest a pathway for linking the teaching of software engineering project with the teaching of programming.

2 RELATED WORK

Capstone Software Engineering Projects. Project-based software engineering courses are essential for students to get training for professional software development skills like architecture design, team management, software maintenance, etc. Students are often required to work as a team to develop software either from industrial partners or simulated real-world topics via semester-long projects [5, 7, 9, 12, 22, 23]. However, there exist certain barriers and challenges to this teaching setting. For example, continuously collecting project topics from industry partners and establishing an efficient communication channel between stakeholders (students and company clients) are challenging tasks for the instructor. More importantly, the students work on different project topics each year, which means they usually do not have a general picture of the entire system, therefore they cannot experience the evolution of a software system.

In this work, our focus is presenting the idea of having an in-house, long-running, sustainable software engineering project in the university context. This kind of long-running SE project shares characteristics with other community-driven course concepts [3]. Our proposed teaching concept is however novel in the sense that it links the teaching of software engineering courses and the teaching of introductory programming courses. This is done by developing an intelligent tutoring system. Students not only get training for software development but also gain exposure to the latest research in the software engineering community.

Automated Program Repair for Feedback Generation. Automated program repair (APR) [14, 16, 18] is a technique that is designed to automatically provide program patches to reduce developers' manual debugging burden. Prior research [26] has shown the possibility of applying APR techniques in introductory programming courses. Over the last decade, a number of CS-1 specific APR tools have been introduced to rectify programming mistakes and provide feedback for novice programmers. AutoGrader [21] takes in a reference solution and manually-curated program error model to automatically synthesize patches for common mistakes in students' incorrect programs. Clara and SarfGen [10, 24] assume the availability of multiple reference solutions and repair students' incorrect programs at the basic-block level by editing students' faulty statements with expression ingredients from reference solutions. Refactory [11] addresses the assumption of having multiple reference solutions by using refactoring rules to automatically produce more semantically equivalent but syntactically different reference solutions based on a single reference solution. Verifix [1] aims to

improve the trustworthiness of generated patches and performs program equivalence verification to guarantee the correctness of the generated feedback. AssignmentMender [15] leverages students' faulty submissions as patch ingredients to improve the repair rate. There are also works specifically designed for repairing syntax issues (such as compilation errors) in students' submissions [2, 25], whereas our ITS primarily focuses on the repair of programming errors made by the student.

Despite these tools having shown promising results in CS-1 teaching [13, 26], their research outcomes have different focus that cannot be best utilized in a single system. Our Intelligent Tutoring System represents an evolving platform that can integrate the latest research results in APR for education.

3 EXPERIENCE IN SOFTWARE ENGINEERING TEACHING

In this section, we first discuss our teaching concept at a high level (Section 3.1), then share our course arrangement of CS3213 Foundations of Software Engineering at the National University of Singapore in detail for others to adapt (Section 3.2).

3.1 Teaching Concept

Our teaching concept combines lectures about the *foundations* of software engineering with hands-on *project* experience. The goal is to deepen the understanding of software engineering and practice the already learned principles in a realistic environment. In the lectures, we teach foundations with a focus on requirements, modeling notations, software architecture and design, software testing, debugging, and foundations of static program analysis. We also teach non-functional properties like performance and security aspects of software (e.g., software timing analysis and taint analysis). The project focuses on contributing to a more significant, long-running software engineering project to allow the students to go beyond programming-in-the-small. The project aims to develop a functional and ready-to-use intelligent tutoring system incrementally.

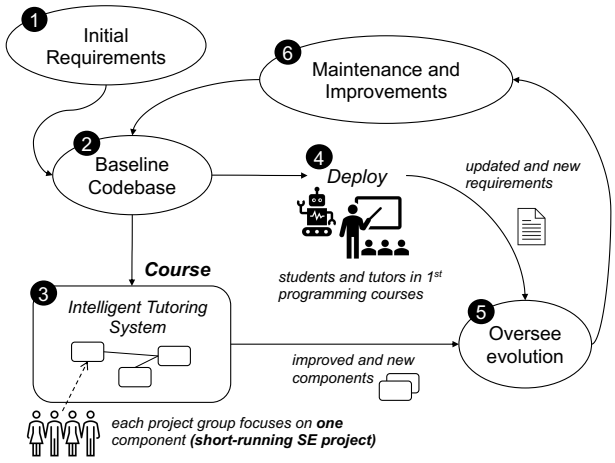


Figure 2: Concept of a long-running software engineering project that is incrementally improved by short-running projects inside a teaching environment.

Developing such a system in the context of a SE course is particularly interesting because (1) the third-year students who develop the system can relate to the users (first-year students) since they once had to learn programming, and (2) all project stakeholders are available in the university. The presence of stakeholders allows embedding requirements elicitation as part of the SE project class.

Figure 2 illustrates the project evolution over multiple years. Before we started any development, we collected initial requirements (step 1) from the lecturers of some first-year programming courses. In step 2, we developed a baseline codebase, which included designing the artifact and the desired workflow. This first version already defined interfaces between components and provided common data structures. The baseline also included a prototypical implementation for most of the initially planned components to test their feasibility. Having the baseline provides the students with additional requirements like the existing architecture, which should not be changed. On the other hand, it also provides them with existing functionalities similar to a real-world long-running SE project.

For our third-year course (step 3), we design multiple short-running SE projects based on the feedback from first year course instructor in the requirement elicitation session of our course, and these projects essentially representing the implementation variants of existing or new components. For example, in the first year, we mainly had projects to build program analysis capabilities. We further designed projects to extend core features like *Automated Feedback* and *Automated Grading* in the second year.

After our course, we evaluate all projects and integrate the best contributions into our baseline implementation (step 5). Therefore, over the years, the baseline will grow and improve. At the same time, we also deploy the increments of the system in its real-world context, i.e., with users from first-year programming courses, and collect additional feedback and requirements from students and tutors (step 4). To keep the implementation standards high and to ensure that our architecture and design can cope with the increasing codebase and the possibly new and changing requirements, we constantly maintain and improve the implementation (step 6).

Overall, our SE project course is structured so that the teaching of SE projects is accomplished over multiple years via a real-life SE project. This project builds an intelligent tutoring system for teaching programming. In the first offering of the course, many system components are not entirely built yet, but these components get built and improved by offering the course over multiple years.

3.2 Course Curriculum

The course curriculum focuses on the main activities in SE. Furthermore, we introduce selected relevant SE topics for our project, e.g., automated program repair, static analysis, and fault localization. Each lecture is separated into two parts: (a) the teaching of foundations in the aforementioned areas, and (b) the teaching of project-specific knowledge and corresponding applications.

Requirements Analysis and Modeling. The course starts with a focus on requirements engineering, their elicitation, and modeling. Therefore, we invite stakeholders like lecturers and teaching assistants from the first-year programming courses to an interview session with the third-year students. This interview session is prepared with corresponding assignments about question design and

Table 1: Course assignment overview that accompanies the major project milestones.

ID	Topic	Details
1	Requirements Analysis & Elicitation	Preparations and questions for the interview session with the customer.
2	Requirements Modeling	Requirement modeling with UML Use Case and Activity diagrams.
3	Architectural Drivers and Architecture Variants	Discussion of architecture variants and the requirements that influence architectural design.
4	Strategy and Project Planning	Project-specific planning including a Gantt-Chart and a resource plan.
5	Detailed Design	Structural and behavioral design of the students' implementation with UML models
6	Intermediate Deliverable	Towards the middle of the course, we ask the students to submit a minimal project implementation and a report with their project plans and various models.
7	Validation (i.e., Unit Testing)	Test case design and test report.
8	Presentation & Final Artifact	At the end of the course, all teams need to present their project and submit their code.
9	Final Report	After the presentation, the students additionally need to submit a final report, including a retrospective of their project and design decisions.

followed up with requirements modeling exercises using UML Use Cases. We also teach other means for requirements modeling, e.g., with finite state machines and sequence diagrams.

Software Architecture and Design. Afterwards, we introduce general principles for software architecture design and modeling. The project-specific part of the lecture introduces the existing architecture and its components, including the available interfaces, which need to be used by the students in their own implementations. We further discuss architecture variants of the existing architecture to discuss pro and contra of the made design decisions.

Our baseline Java implementation already provides the students with elementary classes and functionalities, which they can and need to reuse. To illustrate the fine-grained design, we first introduce relevant design principles and patterns that occur in our implementation. We do not give a comprehensive introduction to design patterns because there is another dedicated software design course in our institution. Instead, we only introduce the most relevant design aspects to enable the students to work on the projects.

Project Planning and Implementation. As part of the assignments, the students have to submit a project plan. Therefore, we also introduce the basics of project planning, work package design, and milestone and resource planning, including necessary models like Gantt-Charts. The coding itself is a major part of the project and is mostly supported by the mentors in project-specific meetings. The lecture introduces general principles like Clean Code and testing and debugging techniques meant to help the students in their concrete implementation efforts.

Testing, Debugging, and Integration. As automated testing and debugging is a major part of an intelligent tutoring system, we also introduce several validation concepts and debugging techniques. In particular, we teach foundations in test-suite estimation, functional testing, whitebox testing, structural testing, dataflow testing, and mutation testing. To this end, we also introduce the basics of static analysis like control-flow graphs (CFGs) and Define-Use Analysis (DUA). Furthermore, we discuss the basics of debugging with the TRAFFIC principle and delta debugging and dive deeper into the basics of static and dynamic slicing and statistical fault localization. Towards the end of the curriculum, we also discuss integration testing strategies and the related challenges.

Project-Specific Topics. In addition to the foundations in general software engineering, we teach the background in automated program repair and provide an overview of existing solutions for ITS components. Depending on the advertised projects, we also discuss more specialized topics like taint analysis and Worst-Case Execution Time (WCET) analysis to ensure the students have the relevant background and material to work on their projects.

Labs and Assignments. Each week in our curriculum is accompanied by a lecture and a lab session. The labs are used to meet in smaller groups of students and discuss their assignments. The assignments track the major milestones in the students' projects (see Table 1).

Team Management. We ask the students to form groups of 3-4 people to work on the project. We allow them to search for their team members instead of a random assignment by the teaching team. We prepare an ungraded *Assignment 0* for the project selection, which provides an overview and additional references for all available projects for the specific year. Each team can bid for three projects, while the teaching team allocates the final project. We encourage each team to join the same lab sessions to maximize the possibility of team interaction. Additionally, each team meets weekly with a graduate-level mentor focusing on the team's planning, design, and implementation progress. The mentors also have access to the team's code repository to provide feedback.

Our teaching materials can be found at:
https://cs3213-fse.github.io/lecture.html

The course aims to enable the students to advance their skills in software development and grasp a deeper understanding of fundamental SE concepts. All student projects eventually contribute to an intelligent tutoring system, whose details are discussed in the following section.

4 INTELLIGENT TUTORING SYSTEM (ITS)

The architecture and design of our intelligent tutoring system (ITS) are inspired by existing research [1, 10, 11, 24] in this area. Figure 3 shows the overview of components and the intended workflow. All components provide interfaces so that components can be implemented independently. In the following, we discuss each component's purpose and illustrate the workflow. For our example, we use the program in Figure 4a as the reference implementation that the lecturer would provide. It is the solution for a simple assignment that requires reading a number n from the console and compute $\sum_{i=1}^{n} \sum_{j=1}^{i} j$, which then should be printed again on the console. Figure 4b shows an incorrect student submission with an error in the loop condition in line 7. In addition to the programs, we also

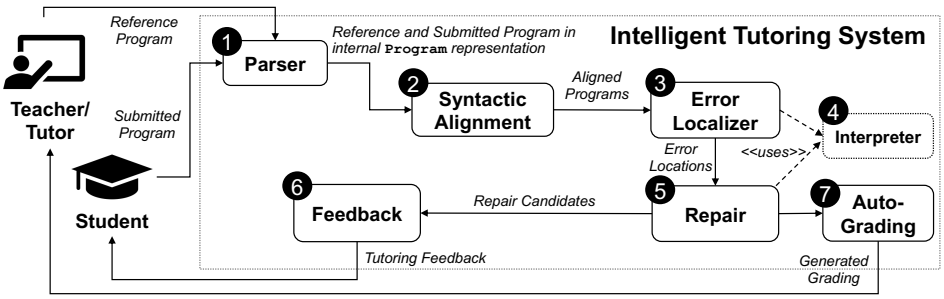


Figure 3: Illustrates the general workflow of the Intelligent Tutoring System.

assume to have some tests, which can be used to assess the correctness of the student submission. In this example, we only have some concrete inputs of interest $n \in \{2, 4, 10, 3, 1, 20\}$, and the correct behavior can be extracted from the reference implementation.

1) Parser. The reference program and the submitted (incorrect) program are given to the *Parser* component. For each program, it generates the corresponding internal program representation. This representation is based on the Control-Flow Graph (CFG). Finally, the results are passed to the *Syntactic Alignment* component. The objective of the *Parser* components is to enable the other internal parts of the Intelligent Tutoring System to work independently from a specific programming language. The simplified illustration of the internal CFG-based program representation for the reference program is shown in Figure 4c. Note that the (incorrect) student submission has the same structure, i.e., the same number of basic blocks, although a different content in the blocks.

2) Syntactic Alignment. The *Syntactic Alignment* component takes the two Program objects and identifies the matching basic blocks. Therefore, it aligns the two programs with regard to their CFG representation. Moreover, it maps the existing variables for each function inside the programs. The results can later be used to detect (error) locations where the reference and submitted programs behave differently. Additionally, this information helps to attempt the repair/fix of the submitted program by reusing information from the reference program. Our current baseline implementation follows the approach by Clara [10], which attempts to match the two programs based on their control flow and their variables. In our example, the structure is the same, so the mapping is straightforward. Internally, we keep a mapping for each function and its basic blocks:

$$main : \{1 = 1, 2 = 2, 3 = 3, 4 = 4, 5 = 5, 6 = 6, 7 = 7\}$$

To build the variable mapping, we use a Define-Use Analysis (DUA) (also see Hu et al. [11]). The resulting variable matching is:

$$main = \{i = i, sum = sum, j = j, n = N\}$$

3) Error Localizer. The *Error Localizer* component identifies locations that show erroneous behavior in the submitted program. This enables others components to formulate a repair/fix. The *Error Localizer* component has access to the *Interpreter* component to execute test cases while observing the values of variables at specific locations. It can use the *Interpreter* to detect semantic differences between the reference and submitted programs. For our example, we use a trace-based error localizer. It uses the *Interpreter* to execute the inputs for both programs and compares the resulting execution traces. For the input $n = 2$, our *Error Localizer* identifies a value mismatch at location 4. It also detects which variable or expression holds the first observation of this mismatch: the loop condition in line 7 in the student's submission, `j<=N`.

4) Interpreter. The *Interpreter* component allows the execution of a program in its CFG-based representation without any compilation or execution on the actual system. It generates an execution trace with the sequence of executed basic blocks and a memory object, which holds the variable values at specific locations.

5) Repair. The *Repair* component attempts to fix the submitted program. For example, it can use the mapping to the reference program (see step 2) and the identified error locations (see step 3) to generate so-called local repairs that modify single statements in the submitted program. Multiple local repairs can be combined to represent more complex changes. The repair process results in a list of plausible repair candidates. It can also use the *Interpreter* component to extract more information from the (correct) reference program. For our example, we use an ILP-based repair implementation similar to [10]. It uses the reference implementation information to search for a minimal change to transform the student's program into the reference program. The local repair with the smallest repair cost is to change the condition location 6 from `j <= N` to `j <= i`. Additionally, we already integrated other repair strategies like Refactory [11] and particularly allow the use of multiple reference implementations to maximize the chances of structural matching between the student's submission and the reference solution.

6) Feedback. With all the collected intermediate information from previous components, the *Feedback* component generates a natural language explanation to guide students to correct their mistakes.

7) Auto-Grading. The *Auto-Grading* component integrates recent research on the concept graph of CS-1 programming assignment [8]. More details appear in the following section.

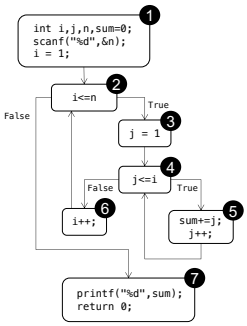
Note that all these components can and are developed in various variants. For example, our current baseline implementation includes a C and Python *Parser* to support multiple languages. The *Error Localizer* can compare execution traces for concrete inputs or perform statistical fault localization. The *Repair* component can follow various strategies, e.g., an optimization-based repair approach like

```
1     #include <stdio.h>
2     int main()
3     {
4        int i,j,n,sum=0;
5        scanf("%d",&n);
6        for(i=1;i<=n;i++)
7        {
8           for(j=1;j<=i;j++)
9           {
10             sum+=j;
11          }
12       }
13       printf("%d",sum);
14       return 0;
15    }
```

(a) Reference Program

```
1     #include <stdio.h>
2     int main(){
3        int i,j,N,sum;
4        sum=0;
5        scanf("%d",&N);
6        for(i=1;i<=N;i++){
7           for(j=1;j<=N;j++){
8              sum=sum+j;
9           }
10       }
11       printf("%d",sum);
12       return 0;
13    }
```

(b) Incorrect Student Submission



(c) Simplified illustration of the CFG-based internal program representation of the code in Figure 4a.

Figure 4: Listings and CFG used in the example to illustrate the workflow in Figure 3.

in [10] or a synthesis-based approach like in [11]. Therefore, the platform we are developing is not only interesting for educational purposes but can also integrate new research advances.

5 STUDENT FEEDBACK AND GRADING

In this section, we elaborate on the key student-facing functionalities of the intelligent tutoring system:

- providing *feedback* to struggling student attempts, and
- *auto-grading* of student assignments.

Student Feedback. The ITS provides feedback to student programming attempts. The feedback is a repair of the student program, vis-a-vis the reference solution, based on the patch locations and candidates produced by repair engines (e.g., Clara and Refactory [10, 11]) in our *Repair* components. We initially started with a pattern-based approach, where the *Feedback* component translates the error type and error location using pre-defined feedback templates. For the example in Figure 4b, the ITS produces the following feedback to give hints on the incorrectly used loop condition. We intentionally hid the fix patch because we aimed to help the students think instead of explicitly showing the answer.

> * *At line 7: Error with loop condition. Wrong variables in the condition. Variables [i, j] should be checked in the loop condition.*

However, the pattern-based approach may not be comprehensive enough and is limited when handling sophisticated errors. Therefore, we incorporate LLM to curate organized human-readable feedback because of their promising performance in text generation [6] in the second year of our course. We provide the error locations and error types generated by *Repair* component as prompt ingredients and follow a few-shot prompting strategy [4] to tune the LLM (e.g., ChatGPT) to produce feedback as a human tutor.

Automated Grading. Test-suite based automated grading suffers from the problem that a small mistake by the student can cause many test cases to fail. To provide better support for tutors, we integrate an auto-grading capability, which aims to test the *conceptual* understanding of the student and awards grades accordingly [8]. This is achieved by constructing a concept graph from the student's

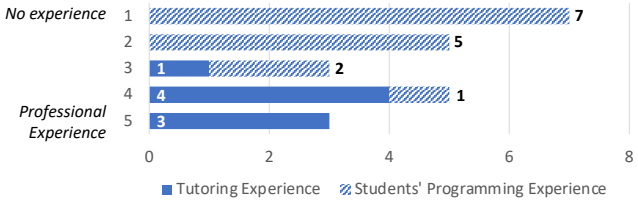


Figure 5: Participants' Self-Assessed Experience

attempt and comparing it with the concept graph of the instructor's reference solution. The aim is to automatically determine which of the ingredient concepts being tested by the programming assignment are correctly understood by the student. Given the instructor-provided reference solutions and students' incorrect solutions, we apply the abstraction rules to convert students' concrete implementation to conceptual understandings and compare them against the conceptual requirements in reference solutions. Based on the result, the *Auto-Grading* component generates a grading report for the tutor. It assesses the student's submission by their missing or improperly used programming concepts to address the over-penalty issue [8] in the conventional test-based assessment.

6 EXPERIENCE IN CS-1 TEACHING

After a year of course iteration and system development, we successfully developed an end-to-end ITS that automatically generates customized feedback for students and tutors. In this section, we share our initial experiences of using the ITS from the viewpoint of our customers – students and tutors based on two institutional review board (IRB)-approved studies.

6.1 Study Methodology

Participants Background. In total, we recruited 15 students and 8 tutors from the CS-1 programming courses at our institution. The students are all from the same programming course. We scheduled the study in the middle of this course so that the students already obtained fundamental programming knowledge but are still in the progress of learning programming. Therefore, we have an interesting scenario where the ITS is deployed to students with little

programming experience (see Figure 5). Our tutor participants are senior undergraduates who have prior experience of tutor duty for CS-1 programming courses at least once. All participants were compensated with 10 SGD.

Study with Students. The student study focused on understanding how the ITS can help students. Therefore, we conducted a controlled experiment and, based on their experience (see Figure 5), we equally divided the student participants into two groups, A and B. The participants are instructed to solve programming tasks using an institution-internal submission system that allows them to run provided test cases. For each task, they had 20 minutes and were allowed to make any number of submission attempts. Additionally, group A had access to the ITS, i.e., these students were able to receive additional feedback. Before they started with the programming tasks, we briefly introduced the ITS to ensure they could use it. Overall, the study was structured in three parts: (1) a background survey, (2) the programming tasks, and (3) a feedback survey. Through parts 1 and 3, we collected additional expectations and feedback for the ITS. So that the students from group B also can provide feedback on the idea of an ITS, we provided group B with a brief introduction to ITS *after* they solved their programming tasks. After the study, we clarified any related questions for both groups.

Study with Tutors. The tutor study focused on understanding the needs of tutors and their feedback on the current deployment of the ITS. The study was structured in three parts: (1) a background survey, (2) the grading of programming tasks with the help of the ITS, and (3) a structured interview about their experience. After part 1, we also provided tutors with a brief introduction to the ITS.

Programming Tasks. We have chosen four entry-level programming tasks covering various programming topics. Table 2 shows the details of each task and their respective topics. We selected these programming tasks for two reasons: (1) they were taken from past mid-term exams of the CS-1 course, which represent the practical challenges students may face, and (2) they followed the weekly course curriculum, which teaches new programming concepts.

Table 2: Subjects of programming tasks in our surveys

Tasks	Description	Topic
Remove Extras	Remove duplicate elements from tuple	For loop, Tuple manipulation
Reverse String I	Iteratively reverse a string	For loop, String manipulation
Reverse String II	Recursively reverse a string	Recursion
Reverse Numbers	Iteratively reverse an integer	While loop

6.2 Result Analysis for Students

We recorded the submitted solutions and their timestamps for each programming task of non-duplicate students' attempts. Students were considered to have *solved* a task if their attempts passed all test cases. In total, we received 128 attempts for the four programming tasks; 65 by Group A and 63 by Group B. For all open-ended questions, we conducted a qualitative content analysis coding [19] that summarizes the themes and opinions. First, one author performed the analysis and coding steps; afterward, another author reviewed them. Finally, after a discussion, we completed the analysis.

All data, survey forms, and experiment artifacts are available: *https://doi.org/10.5281/zenodo.7870623*

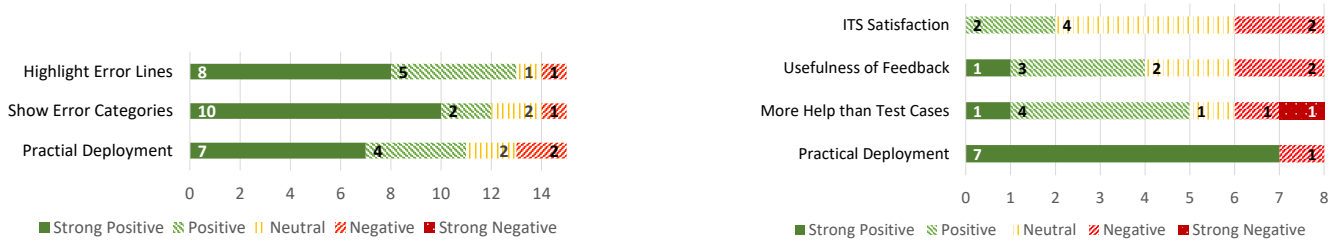
Students' Expectations. Based on the Background (Part 1) survey, we identified the main challenges for novice programmers and their expectations for an ITS. Their general underlying difficulties in learning programming are (1) understanding programming tasks and starting to program, (2) debugging the code and rectifying identified errors, (3) translating their solution strategy into actual code, (4) having trouble with the syntax of a specific programming language, and (5) getting the program right in the first place. In addition, we asked the students more specifically about the difficulties the ITS can address. Generally, they confirmed that their main difficulties are with (1) figuring out what goes wrong in the program and (2) finding the error location. Only half of them (7/15) mentioned that identifying a fix is a problem.

Students' Performance. Table 3 presents the quantitative results of the students' performances in the two controlled groups. Specifically, we focus on students who failed on their first attempt. The second column represents the average number of students' attempts for each task if their first attempt failed. The third column represents the rectification rate (X/Y) of students who failed to solve a particular task on the first attempt; X represents the number of students who eventually rectified their solutions, and Y represents the number of students who failed to solve a task on the first attempt. The column "*Avg Rectifying Time*" indicates the duration taken by a student to correct an incorrect solution for a programming task.

Table 3: The average number of failed attempts, rectification rates, average rectifying time of failed attempts in minutes.

Tasks	Avg # Failed Attempts		Rectification Rate		Avg Rectifying Time (mins)	
	A	B	A	B	A	B
Task 1	4.8	4	4/5	0/2	7	-
Task 2	1.9	5.5	7/7	3/4	9.2	9.3
Task 3	2.3	2.8	5/5	2/4	4.6	2.5
Task 4	2.3	3.1	5/6	5/7	4.5	11.3
Total	2.7	3.7	21/23	10/17	6.7	8.9

Fewer attempts, higher accuracy. As shown in Table 3, students who received assistance from ITS (Group A) solved more programming tasks with fewer attempts compared to students without ITS (Group B). Although Group A made more attempts than Group B for Task 1, it is important to note that the two students in Group B who failed Task 1 could not rectify their solution. Therefore, the fewer average attempts made by Group B may be due to a lack of knowledge on how to fix their solutions after a few attempts, resulting in giving up on the task. On average, Group A made 2.7 failed attempts compared to 3.7 for Group B, indicating that Group A submitted slightly fewer attempts during the experiment. Even though the difference in attempts is not very significant, Group A had a higher success rate in rectifying their solutions; they successfully fixed 21 (91.3%) out of the 23 failed attempts. While Group B had a higher



(a) Students' feedback of ITS

(b) Tutors' overall satisfaction with ITS.

Figure 6: Results from Study Surveys and Interviews.

success rate on their first attempt, they struggled more when they failed on their first attempt, only succeeding in fixing 10/17 (58.8%), demonstrating the effectiveness of ITS guidance.

Regarding rectifying time, Group A was faster, with an average of 6.7 minutes to repair one incorrect solution, compared to Group B's average of 8.9 minutes. The average rectifying time for task 1 in Group B is unavailable since no student could rectify their incorrect attempts. Moreover, the average rectifying time for Group B is significantly lower for Task 3 (2.5 minutes) because the two incorrect solutions were almost correct (e.g., typos in function names). Our supplemental material includes more details.

Usefulness of ITS. Figure 6a shows feedback survey results for students, where we queried their satisfaction with the ITS regarding the usefulness of the features, such as highlighting the potential error lines in the code editor and showing hints about error categories for their mistakes. The results of the questions indicated that most Group A students found the ITS helpful and were satisfied with its feedback and current shape. For example, over 80% of the students responded positively to the usefulness of highlighted lines and mistake categories for their code. Furthermore, over 73% of the students would like the ITS deployed in their programming course. However, we found that one student showed negative feedback toward all questions. This student failed to solve any tasks with correct syntax and struggled to find proper solution strategies. As a result, the ITS could not generate any feedback, as it could not explain the student's intuition at this stage. While this particular experience highlights the limitations of the ITS, the overall positive feedback from the other students supports the potential of ITS in enhancing CS-1 programming education.

Students' Performances: The collected performance results support that the ITS indeed helps the students to take *fewer* attempts to solve *more* programming tasks in a *shorter* time.

6.3 User Evaluation Result for Tutors

Tutors' Expectations about the ITS. Like the students, we asked tutors to identify their general difficulties and expected features. They are primarily concerned about generating the actual feedback for students, for which they first need to identify the semantic and syntax errors in the submission. Generating high-quality feedback becomes increasingly difficult because of the large number of submissions that tutors have to handle. Error localization techniques are helpful as they can help tutors pinpoint erroneous code areas faster than going through the complete submission themselves. Once identified, fixing the actual error or grading the overall submission is less of a problem for them. Therefore, the most helpful feature from the already implemented ones also mentioned by the students is the error localization and explanation. Although we observed the students' demand for addressing compilation/syntax mistakes, tutors showed low interest in compilation error repair, which shows the discrepancy in requirements between tutors and students. Tutors prefer feedback on semantic over syntactic errors.

Tutors' Expectations: Tutors mostly expect support in error localization and explanation, and need less support for fix suggestions, particularly for syntax errors.

Tutors' Satisfaction with the ITS. We interviewed the tutors after they finished the grading activity of student submissions in Section 6.2. The result shows that for all tutors, the ITS's currently available error localization capabilities are indeed most beneficial for them. Most of them mentioned that the ITS helps them to speed up their work in understanding the student's errors (5/8) and formulating their own feedback (6/8). However, none of them found that the feedback given by the ITS can be *directly* used or can directly determine the mark of the student. Note that the ITS had no automated grading capabilities at the time of the study. Overall, the participants presented varying opinions about the usefulness of the feedback (see Figure 6b). The tutors noticed that the given feedback is biased towards only having one reference solution: the suggested feedback/repair of a submission can be non-minimal if the student submission follows a different strategy. We plan to address this limitation by fully integrating the approach by Refactory [11], which generates additional semantically equivalent but structurally different reference solutions via refactoring. The overall satisfaction with the tool is diverse, mainly ascribed to the limited feedback capabilities. Still, almost all of the tutors (7/8) would like to see an ITS deployment in their next programming course, primarily because of the strong support for error localization, which provides a good starting point for understanding the student's problem and grading. Furthermore, most of the tutors (5/8) see the additional help by the ITS as an improvement on the already available information from failing test cases.

Tutors' Experiences: The tutors reported that the ITS can help them handle their grading tasks faster via its automated error localization capability. Despite their concerns regarding the current feedback and grading capabilities, they identified ITS as helpful and would like to use it in their next CS-1 course.

7 DEPLOYMENT EXPERIENCE

In addition to the observations from the conducted user studies, we also gained experience by integrating the ITS into the Python introductory programming course CS1010S at the National University of Singapore. As a first step towards a complete deployment, the current deployment generates feedback visible only to the human tutors to support their manual feedback and grading efforts. We share the experience of deploying the ITS in CS1010S throughout the fall semester of the 2023 – 2024 academic year, which involved 571 students. CS1010S covers the programming topics of recursion, higher-order functions, abstract data types, file processing, basic data structures like binary search trees and sorting algorithms, and object-oriented programming via nine weekly assignments. In CS1010S, students attempt to solve the programming assignments and submit their solutions in an online programming platform called Coursemology. It automatically evaluates the submitted solutions by running a set of pre-defined test cases. Students may make multiple attempts to revise their solutions before the deadline. After the submission deadline, tutors manually review the incorrect submissions that fail the private test cases and compose feedback comments for the students.

We integrated our ITS with Coursemology and deployed it for 30 programming tasks spanning six weeks of programming assignments. We excluded the first two introductory weeks and the final week, which involves object-oriented programming that the current ITS does not support. In addition to the traditional CS1010S workflow, Coursemology invokes the ITS to automatically generate feedback when a student submits a solution that fails public tests. The generated feedback consists of (1) the location of students' mistakes and (2) a correct patch that fixes students' mistakes. The feedback is available for the tutors to reduce their manual effort in understanding the students' mistakes and composing the feedback. We tracked the ITS-generated feedback for all incorrect student submissions during the semester and share our deployment experience of ITS in a live CS-1 course.

Scalability and Effectiveness of ITS. For a total of 3117 incorrect solutions submitted by 553 students throughout the semester, the deployed ITS successfully generated 1758 (56.4%) feedback comments with correct patches. As the main reasons for failing to generate feedback, we identified that (1) the students' solutions either lacked substantial content or (2) violated specific assignment requirements. In the first case, we conclude that students could not find the correct solution strategy, so they gave up by submitting an empty file or attempting to brute-force the public tests. In the second case, the submitted solutions passed all test cases. Still, they violated an additional assignment requirement, which was not covered by the tests and instead needed to be manually checked by the tutors. For example, in an assignment about abstract data types, the students had to reuse a specific abstract function; failing to do so led to additional penalties, which the ITS cannot detect. Additionally, we randomly sampled 10% of ITS's feedback and the corresponding tutors' feedback. Then, we manually checked whether they were semantically equivalent to evaluate the quality of ITS's automated feedback. Figure 7 shows an example of two semantic equivalent feedback given by ITS and the tutor for a student's mistake of not handling an edge case (lines 7 – 8 show the patch generated by ITS). Our analysis shows that 136 (77.2%) of ITS-produced feedback is semantic equivalent to tutors' feedback, illustrating its capabilities to generate high-quality feedback.

```
1   def find_transcription_region(dna_strand):
2     if not "TATA" in dna_strand or not "CGCG" in
          dna_strand:
3       return None
4     else:
5       tata_box_index = dna_strand.find("TATA")
6       stop_sequence_index = dna_strand.find("CGCG")
7   +++ if tata_box_index > stop_sequence_index:
8   +++     return None
9       return dna_strand[tata_box_index+len("TATA"):
            stop_sequence_index+len("CGCG")]
10  -------
11  #ITS's Feedback:
12  The student's program does not handle the case where
        the TATA box appears after the stop sequence CGCG
13  #Tutor's Feedback:
14  [-1] doesn't check if TATA is before CGCG
```

Figure 7: Example of two semantic equivalent feedback given by ITS and tutor for missing the edge case at lines 7–8.

Deployment Experience: The first ITS deployment shows that the ITS is scalable in a large course setting, it is effective in fixing mistakes made by CS-1 students, and more than 77% generated feedback is aligned with tutors' feedback. However, some limitations need to be addressed in the future: the current ITS (1) is not at the stage of providing step-by-step guidance to help students solve assignments from scratch, and (2) cannot provide high-level feedback for quality aspects beyond the functionality.

Organizational and Technical Deployment Aspects. During this deployment, we also experienced organizational and technical challenges that may affect the broader deployment of the ITS. For example, each deployment requires the careful adaption of the ITS to the specific teaching setup of the various CS-1 programming courses. This involves the programming language, the support for specific assignment types (e.g., GUI programming), or integrations into additional learning management platforms. These are particularly important because the expectations and pedagogical strategies differ for students in various departments. We also encountered technical issues regarding user interaction. For example, many students used to submit assignments close to the deadline, which caused a high feedback generation demand in a short period. The unexpected task traffic may overload the server and thus cause an unpleasant user experience. Future instances of CS3213 would benefit from projects focusing on building a robust front-end client for instructors and students. To achieve practical impact and ensure a great learning experience, such interaction-related aspects are equally important compared to the core back-end functionalities of the ITS, which we have mainly focused on in the past two years.

8 CHALLENGES & LESSONS LEARNED

To further share our experience with our combined research and teaching effort, we report the challenges we faced and the lessons learned concerning the teaching of our third-year course.

Incentives for Stakeholders. We have three main user groups: the students who receive feedback, the tutors who can use the ITS to better understand the students' errors and get grading support, and the lecturers who provide the inputs like assignments and reference implementations. *Lecturers* are naturally concerned about deploying more tools, including the potential negative effects on the learning outcome caused by inaccurate output. To gradually convince the lecturers, we decided to first focus on a targeted deployment for tutors. For tutors, an imperfect output is less critical and still can provide helpful guidance to them and help us to get feedback continuously. In contrast to the lecturers, the *tutors* have a generally more positive attitude regarding the ITS; they are willing to join longer interviews to share their experience in the tutoring process and their requirements. As a result, we have been able to successfully invite tutors to our requirements elicitation sessions as well as to our user studies. To engage with *first-year students*, we designed a user study that not only has a monetary reimbursement but also provides additional programming training and an extra tutorial after the user study to explain the programming tasks to them individually. The *third-year students* who develop the components in our course showed great interest in our project because it is (or will be) deployed in a real context and because they like working on a larger project with existing parts. Overall, it is a valuable experience for them, as shown by the following student quotes about the question of what they liked the most in the course:

"The project component – It's really interesting, and I like that it will actually be used. I think that makes it one of the most interesting modules I've taken so far. It's very cool to understand the reasoning for design details with the teaching team that actually built it."

"Participation in an actual to–be–deployed software project is exciting and makes your effort somewhat worthwhile."

Project Preferences. In the first instance of our course, we allowed students to pick projects on their own. Therefore, we ended up with an imbalanced selection of projects. Students tended to prefer a project with more explicit requirements, e.g., a *Parser* component, instead of a *Repair* project that involves more research. In the second instance, we therefore only allowed bidding on projects while the teaching team made the final decision.

Mentoring support for third-year students. In the second instance of our course, we had dedicated, experienced mentors (i.e., graduate students) who helped the student groups organize their efforts. While we did not observe the students without mentors in our first-course instance perform poorly, we still experienced that the additional mentorship helped them get the best out of their project. This is not only helpful to improve our system but also creates a better project experience for them.

Managing Software Evolution. Overall, we experienced that our general approach is feasible and helps both the third-year and the first-year students. However, we have also seen that we must invest significant time from our side in managing the software evolution. This includes selecting and integrating the best projects, maintaining the code base, updating the design to cater to new requirements, and implementing new components to check their feasibility before we can offer them as a project in the course.

9 IMPACT AND VISION FOR THE FUTURE

In this work, we presented our concept for linking the teaching of software engineering projects with the teaching of programming and introduced our intelligent tutoring system (ITS). Further, we discussed our experiences using the ITS in our course and the observations from the user studies. In the following two sections, we discuss the observed *impact* of our work and provide a concluding *outlook* for intelligent tutoring in the AI era.

Impact: Teachers, Students, Research. Based on our experience, the presented ITS impacts several aspects of programming. With our long-running teaching effort, we incrementally develop and improve the ITS into a usable product. We change how first-year *students* learn programming and support *teachers* in the introductive CS courses. Furthermore, we provide the platform for senior *students* to practice software engineering in a realistic scenario. Additionally, they are encouraged to work on research-oriented topics by selecting the corresponding projects. Overall, we received positive feedback in our user studies: from the 23 students and tutors, more than 78% would like to see the ITS deployed in their next programming course. Moreover, the ITS helps to integrate the latest *research* in educational APR and related topics. Our teaching innovation can also impact students from other universities as they adopt our concept and join the ITS development team. In fact, we have already exported the teaching concept to another university.

Intelligent Tutoring in AI Era. With the shift from manual programming to AI-assisted programming, CS education must also be innovated. We think the ITS represents a well-suited platform to help students learn an effective way of using AI-based code generation tools like GitHub Copilot and ChatGPT. Therefore, instead of exposing the student directly to the AI assistant, the ITS can *moderate* the prompts and explain the generated code, achieving a three-way interaction between the student, ITS, and AI assistant. Based on the student's performance, mistakes, and interaction with the AI assistant, the ITS can learn a model of the student's current mental model. This can be achieved by mapping the student's mistakes and questions to the underlying programming concepts.

DATA AVAILABILITY

All data, survey forms, and experiment artifacts are available at: *https://doi.org/10.5281/zenodo.7870623*. Our teaching materials can be found at: *https://cs3213-fse.github.io/lecture.html*

ACKNOWLEDGMENTS

We would like to thank Martin Mirchev, Ruan Haifeng, Kishore R, Jon Chua, Guo Ai, Huang Zhenxin, Christopher Goh, Eris Jacey Masagaca, Ethan Noah Rozario, and all students, tutors, and lecturers who have supported our work in two offerings of the course. We thank the anonymous reviewers for their suggestions. This work was partially supported by a Singapore Ministry of Education (MoE) Tier 3 grant "Automated Program Repair", MOE-MOET32021-0001.

REFERENCES

[1] Umair Z. Ahmed, Zhiyu Fan, Jooyong Yi, Omar I. Al-Bataineh, and Abhik Roychoudhury. 2022. Verifix: Verified Repair of Programming & Assignments. *ACM Trans. Softw. Eng. Methodol.* 31, 4, Article 74 (jul 2022), 31 pages. https://doi.org/10.1145/3510418

[2] Umair Z. Ahmed, Pawan Kumar, Amey Karkare, Purushottam Kar, and Sumit Gulwani. 2018. Compilation Error Repair: For the Student Programs, from the Student Programs. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering Education and Training* (Gothenburg, Sweden) *(ICSE-SEET '18)*. Association for Computing Machinery, New York, NY, USA, 78–87. https://doi.org/10.1145/3183377.3183383

[3] Boyd Anderson, Martin Henz, and Kok-Lim Low. 2023. Community-Driven Course and Tool Development for CS1. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (Toronto ON, Canada) *(SIGCSE 2023)*. Association for Computing Machinery, New York, NY, USA, 834–840. https://doi.org/10.1145/3545945.3569740

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[5] Bernd Bruegge, Stephan Krusche, and Lukas Alperowitz. 2015. Software engineering project courses with industrial clients. *ACM Transactions on Computing Education (TOCE)* 15, 4 (2015), 1–31.

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[7] David Delgado, Alejandro Velasco, Jairo Aponte, and Andrian Marcus. 2017. Evolving a project-based software engineering course: A case study. In *2017 IEEE 30th Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 77–86.

[8] Zhiyu Fan, Shin Hwei Tan, and Abhik Roychoudhury. 2023. Concept-Based Automated Grading of CS-1 Programming Assignments. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) *(ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 199–210. https://doi.org/10.1145/3597926.3598049

[9] A. Goold and P. Horan. 2002. Foundation software engineering practices for capstone projects and beyond. In *Proceedings 15th Conference on Software Engineering Education and Training (CSEE&T 2002)*. 140–146. https://doi.org/10.1109/CSEE.2002.995206

[10] Sumit Gulwani, Ivan Radiček, and Florian Zuleger. 2018. Automated Clustering and Program Repair for Introductory Programming Assignments. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Philadelphia, PA, USA) *(PLDI 2018)*. Association for Computing Machinery, New York, NY, USA, 465–480. https://doi.org/10.1145/3192366.3192387

[11] Yang Hu, Umair Z. Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 388–398. https://doi.org/10.1109/ASE.2019.00044

[12] Lynette Johns-Boast and Shayne Flint. 2013. Simulating industry: An innovative software engineering capstone design course. In *2013 IEEE Frontiers in Education Conference (FIE)*. IEEE, 1782–1788.

[13] Oka Kurniawan, Christopher M. Poskitt, Ismam Al Hoque, Norman Tiong Seng Lee, Cyrille Jégourel, and Nachamma Sockalingam. 2023. How Helpful do Novice Programmers Find the Feedback of an Automated Repair Tool? arXiv:2310.00954 [cs.CY]

[14] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2011. Genprog: A generic method for automatic software repair. *Ieee transactions on software engineering* 38, 1 (2011), 54–72.

[15] Leping Li, Hui Liu, Kejun Li, Yanjie Jiang, and Rui Sun. 2023. Generating Concise Patches for Newly Released Programming Assignments. *IEEE Transactions on Software Engineering* 49, 1 (2023), 450–467. https://doi.org/10.1109/TSE.2022.3153522

[16] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the 38th international conference on software engineering*. 691–701.

[17] Samim Mirhosseini, Austin Z. Henley, and Chris Parnin. 2023. What Is Your Biggest Pain Point? An Investigation of CS Instructor Obstacles, Workarounds, and Desires. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (Toronto ON, Canada) *(SIGCSE 2023)*. Association for Computing Machinery, New York, NY, USA, 291–297. https://doi.org/10.1145/3545945.3569816

[18] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. Semfix: Program repair via semantic analysis. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 772–781.

[19] Margrit Schreier. 2012. *Qualitative content analysis in practice.* Sage publications.

[20] Natasha Singer. 2019. The Hard Part of Computer Science? Getting Into Class. https://www.nytimes.com/2019/01/24/technology/computer-science-courses-college.html Accessed: 16-March-2023.

[21] Rishabh Singh, Sumit Gulwani, and Armando Solar-Lezama. 2013. Automated feedback generation for introductory programming assignments. In *Proceedings of the 34th ACM SIGPLAN conference on Programming language design and implementation*. 15–26.

[22] Maria Spichkova. 2019. Industry-oriented project-based learning of software engineering. In *2019 24th International conference on engineering of complex computer systems (ICECCS)*. IEEE, 51–60.

[23] Saara Tenhunen, Tomi Männistö, Matti Luukkainen, and Petri Ihantola. 2023. A systematic literature review of capstone courses in software engineering. *Information and Software Technology* (2023), 107191.

[24] Ke Wang, Rishabh Singh, and Zhendong Su. 2018. Search, Align, and Repair: Data-Driven Feedback Generation for Introductory Programming Exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Philadelphia, PA, USA) *(PLDI 2018)*. Association for Computing Machinery, New York, NY, USA, 481–495. https://doi.org/10.1145/3192366.3192384

[25] Michihiro Yasunaga and Percy Liang. 2021. Break-it-fix-it: Unsupervised learning for program repair. In *International Conference on Machine Learning*. PMLR, 11941–11952.

[26] Jooyong Yi, Umair Z Ahmed, Amey Karkare, Shin Hwei Tan, and Abhik Roychoudhury. 2017. A feasibility study of using automated program repair for introductory programming assignments. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 740–751.

[27] Jialu Zhang, José Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2022. Repairing Bugs in Python Assignments Using Large Language Models. *arXiv preprint arXiv:2209.14876* (2022).